

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский университет ИТМО»
(Университет ИТМО)

О Т Ч Е Т
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 5
«Процедуры, функции, триггеры в PostgreSQL»
по дисциплине «Проектирование и реализация баз данных»

Обучающийся Смирнов Фёдор Евгеньевич
Факультет прикладной информатики
Группа К3240
Направление подготовки 09.03.03 Прикладная информатика
Образовательная программа Мобильные и сетевые технологии 2024
Преподаватель Говорова Марина Михайловна

Санкт-Петербург,
2026

Цель работы

Цель работы — овладеть практическими создания и использования процедур, функций и триггеров в базе данных PostgreSQL.

Практическое задание

В ходе выполнения лабораторной работы требовалось создать и проверить работу процедур, функций и триггеров в базе данных PostgreSQL для индивидуальной предметной области.

1. Создать 3 хранимые процедуры для индивидуальной базы данных согласно варианту 12 «Прокат автомобилей»:
 - выполнить списание автомобилей, выпущенных ранее заданного года;
 - выполнить выдачу автомобиля и расчёт стоимости проката с учётом скидки постоянным клиентам;
 - вычислить количество автомобилей заданной марки.
2. Создать 7 необходимых триггеров для индивидуальной базы данных, обеспечивающих контроль бизнес-правил предметной области.
3. Создать триггерные функции, необходимые для работы разработанных триггеров.
4. Выполнить проверку работы созданных процедур и триггеров в консоли SQL Shell psql.
5. Продемонстрировать результаты выполнения процедур и срабатывания триггеров на данных базы данных «Прокат автомобилей».

В рамках лабораторной работы были реализованы процедуры для списания автомобилей, оформления выдачи автомобиля клиенту с расчётом стоимости проката и подсчёта автомобилей заданной марки. Также были созданы триггеры для проверки корректности операций с договорами проката, продлениями договоров, страховками и штрафами.

Схема базы данных (ЛР3)

- БД: carsharing
- схема: rental
- вариант: 12
- предметная область: прокат автомобилей

SQL-команда создания процедуры.

```
CREATE OR REPLACE PROCEDURE rental.write_off_cars_before_year(  
p_year INTEGER  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
UPDATE rental.car  
SET  
written_off = 'Y',  
written_off_date = CURRENT_DATE  
WHERE production_year < p_year  
AND written_off = 'N';  
END;  
$$;
```

Процедура `write_off_cars_before_year` выполняет списание автомобилей, выпущенных ранее заданного года. Списание реализовано через изменение признака `written_off` на значение `Y` и заполнение даты списания в поле `written_off_date`.

SQL-команда проверки процедуры.

```
CREATE OR REPLACE PROCEDURE rental.write_off_cars_before_year(  
p_year INTEGER  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
UPDATE rental.car  
SET  
written_off = 'Y',  
written_off_date = CURRENT_DATE  
WHERE production_year < p_year  
AND written_off = 'N';  
END;  
$$;
```

В результате проверки автомобили, выпущенные ранее 2023 года, получили признак списания `Y`, а поле `written_off_date` было заполнено текущей датой.

```

carsharing=# BEGIN;
BEGIN
carsharing=#
carsharing=# SELECT
carsharing-#      id,
carsharing-#      vin,
carsharing-#      production_year,
carsharing-#      written_off,
carsharing-#      written_off_date
carsharing-# FROM rental.car
carsharing-# WHERE production_year < 2023
carsharing-# ORDER BY production_year, id;
id |      vin      | production_year | written_off | written_off_date
-----+-----+-----+-----+-----
  3 | XTA00000000000003 |          2021 | N          |
  1 | XTA00000000000001 |          2022 | N          |
  6 | XTA00000000000006 |          2022 | N          |
(3 строки)

carsharing=#
carsharing=# CALL rental.write_off_cars_before_year(2023);
CALL
carsharing=#
carsharing=# SELECT
carsharing-#      id,
carsharing-#      vin,
carsharing-#      production_year,
carsharing-#      written_off,
carsharing-#      written_off_date
carsharing-# FROM rental.car
carsharing-# WHERE production_year < 2023
carsharing-# ORDER BY production_year, id;
id |      vin      | production_year | written_off | written_off_date
-----+-----+-----+-----+-----
  3 | XTA00000000000003 |          2021 | Y          | 2026-05-13
  1 | XTA00000000000001 |          2022 | Y          | 2026-05-13
  6 | XTA00000000000006 |          2022 | Y          | 2026-05-13
(3 строки)

carsharing=#
carsharing=# ROLLBACK;
ROLLBACK
carsharing=#

```

Рисунок 2 — Проверка работы процедуры списания автомобилей, выпущенных ранее заданного года.

1.2. Выдача автомобиля и расчёт стоимости с учётом скидки постоянным клиентам

SQL-команда создания процедуры.

```

CREATE OR REPLACE PROCEDURE rental.issue_car_with_discount(
p_passport_id BIGINT,
p_car_id BIGINT,
p_employee_id BIGINT,
p_issue_datetime TIMESTAMP,
p_rental_days INTEGER,
p_insurance_id BIGINT,
INOUT p_total_cost NUMERIC
)
LANGUAGE plpgsql
AS $$

```

```

DECLARE
v_model_id BIGINT;
v_daily_rate NUMERIC(12,2);
v_discount_percent NUMERIC(5,2);
v_insurance_type VARCHAR(50);
v_insurance_cost NUMERIC(12,2);
v_deposit_amount NUMERIC(12,2);
v_contract_number VARCHAR(20);
BEGIN
IF p_rental_days <= 0 THEN
RAISE EXCEPTION 'Количество дней проката должно быть больше
0';
END IF;

SELECT c.model_id
INTO v_model_id
FROM rental.car c
WHERE c.id = p_car_id
AND c.written_off = 'N';

IF v_model_id IS NULL THEN
RAISE EXCEPTION 'Автомобиль с id % не найден или списан',
p_car_id;
END IF;

IF EXISTS (
SELECT 1
FROM rental.rental_contract rc
WHERE rc.car_id = p_car_id
AND rc.return_datetime IS NULL
AND rc.contract_status IN ('открыт', 'просрочен')
) THEN
RAISE EXCEPTION 'Автомобиль уже находится в незакрытом
договоре';
END IF;

SELECT rr.price
INTO v_daily_rate
FROM rental.rental_rate rr
WHERE rr.model_id = v_model_id
AND rr.start_date <= p_issue_datetime::date
AND (rr.end_date IS NULL OR rr.end_date >=
p_issue_datetime::date)
ORDER BY rr.start_date DESC
LIMIT 1;

IF v_daily_rate IS NULL THEN
RAISE EXCEPTION 'Не найден актуальный тариф для автомобиля';
END IF;

SELECT
i.insurance_type,
i.insurance_cost
INTO
v_insurance_type,
v_insurance_cost
FROM rental.insurance i
WHERE i.id = p_insurance_id

```

```

AND i.model_id = v_model_id
AND i.start_date <= p_issue_datetime::date
AND (i.end_date IS NULL OR i.end_date >=
p_issue_datetime::date);

IF v_insurance_cost IS NULL THEN
RAISE EXCEPTION 'Не найдена актуальная страховка для
автомобиля';
END IF;

SELECT d.deposit_amount
INTO v_deposit_amount
FROM rental.deposit d
WHERE d.model_id = v_model_id
AND d.start_date <= p_issue_datetime::date
AND (d.end_date IS NULL OR d.end_date >=
p_issue_datetime::date)
ORDER BY d.start_date DESC
LIMIT 1;

IF v_deposit_amount IS NULL THEN
RAISE EXCEPTION 'Не найдена актуальная залоговая стоимость для
автомобиля';
END IF;

SELECT
CASE
WHEN cl.regular_customer = 'Y' THEN cl.discount_percent
ELSE 0
END
INTO v_discount_percent
FROM rental.passport p
JOIN rental.client cl
ON cl.id = p.client_id
WHERE p.id = p_passport_id;

IF v_discount_percent IS NULL THEN
RAISE EXCEPTION 'Паспорт с id % не найден', p_passport_id;
END IF;

p_total_cost =
v_daily_rate * p_rental_days * (1 - v_discount_percent / 100)
+ v_insurance_cost;

v_contract_number = 'LR5-' || to_char(clock_timestamp(),
'MMDDHH24MISSMS');

INSERT INTO rental.rental_contract (
passport_id,
contract_number,
car_id,
employee_id,
contract_date,
issue_datetime,
return_datetime,
insurance_type,
insurance_cost,
deposit_amount,

```

```

deposit_returned_full,
deposit_returned_partial,
return_mark,
contract_status,
insurance_id
)
VALUES (
p_passport_id,
v_contract_number,
p_car_id,
p_employee_id,
CURRENT_DATE,
p_issue_datetime,
NULL,
v_insurance_type,
v_insurance_cost,
v_deposit_amount,
'N',
'N',
'N',
'открыт',
p_insurance_id
);
END;
$$;

```

SQL-команда проверки процедуры.

```

BEGIN;

CALL rental.issue_car_with_discount(
1,
4,
1,
'2025-06-01 10:00:00',
3,
1,
NULL
);

SELECT
id,
passport_id,
contract_number,
car_id,
employee_id,
issue_datetime,
insurance_type,
insurance_cost,
deposit_amount,
contract_status
FROM rental.rental_contract
WHERE contract_number LIKE 'LR5-%'
ORDER BY id DESC
LIMIT 1;

ROLLBACK;

```


В результате проверки была рассчитана стоимость проката 14650.00 и создан тестовый договор со статусом открыт.

```
carsharing=# BEGIN;
BEGIN
carsharing=#
carsharing=# CALL rental.issue_car_with_discount(
carsharing(=# 1,
carsharing(=# 4,
carsharing(=# 1,
carsharing(=# '2025-06-01 10:00:00',
carsharing(=# 3,
carsharing(=# 1,
carsharing(=# NULL
carsharing(=# );
p_total_cost
-----
14650.000000000000000000000000
(1 строка)

carsharing=#
carsharing=# SELECT
carsharing(=# id,
carsharing(=# passport_id,
carsharing(=# contract_number,
carsharing(=# car_id,
carsharing(=# employee_id,
carsharing(=# issue_datetime,
carsharing(=# insurance_type,
carsharing(=# insurance_cost,
carsharing(=# deposit_amount,
carsharing(=# contract_status
carsharing(=# FROM rental.rental_contract
carsharing(=# WHERE contract_number LIKE 'LR5-%'
carsharing(=# ORDER BY id DESC
carsharing(=# LIMIT 1;
id | passport_id | contract_number | car_id | employee_id | issue_datetime | insurance_type | insurance_cost | deposit_amount | contract_status
-----
32 | 1 | LR5-0513195455657 | 4 | 1 | 2025-06-01 10:00:00 | КАСКО базовое | 2500.00 | 15000.00 | открыт
(1 строка)

carsharing=#
carsharing=# ROLLBACK;
ROLLBACK
carsharing=#
```

Рисунок 3 — Проверка работы процедуры выдачи автомобиля и расчёта стоимости с учётом скидки постоянному клиенту.

1.3. Вычисление количества автомобилей заданной марки

SQL-команда создания процедуры.

```
CREATE OR REPLACE PROCEDURE rental.count_cars_by_brand(
p_brand VARCHAR,
INOUT p_car_count INTEGER
)
LANGUAGE plpgsql
AS $$
BEGIN
SELECT COUNT(c.id)
INTO p_car_count
FROM rental.car c
JOIN rental.model m
ON m.id = c.model_id
WHERE LOWER(m.brand) = LOWER(p_brand);
END;
$$;
```

Процедура count_cars_by_brand вычисляет количество автомобилей заданной марки. Для подсчёта используется соединение таблиц rental.car и rental.model.

SQL-команда проверки процедуры.

```

SELECT
m.brand,
COUNT(c.id) AS car_count
FROM rental.model m
JOIN rental.car c
ON c.model_id = m.id
GROUP BY m.brand
ORDER BY m.brand;

CALL rental.count_cars_by_brand('Toyota', 0);

CALL rental.count_cars_by_brand('Volkswagen', 0);

```

В результате проверки процедура вернула количество автомобилей марки Toyota — 2, а количество автомобилей марки Volkswagen — 1.

```

carsharing=# SELECT
carsharing-#      m.brand,
carsharing-#      COUNT(c.id) AS car_count
carsharing-# FROM rental.model m
carsharing-# JOIN rental.car c
carsharing-#      ON c.model_id = m.id
carsharing-# GROUP BY m.brand
carsharing-# ORDER BY m.brand;
   brand   | car_count
-----+-----
 Hyundai   |         2
 Kia       |         2
 Lada      |         1
 Renault   |         1
 Skoda     |         1
 Toyota    |         2
 Volkswagen |         1
(7 строк)

carsharing=#
carsharing=# CALL rental.count_cars_by_brand('Toyota', 0);
 p_car_count
-----
          2
(1 строка)

carsharing=#
carsharing=# CALL rental.count_cars_by_brand('Volkswagen', 0);
 p_car_count
-----
          1
(1 строка)

carsharing=#

```

Рисунок 4 — Проверка работы процедуры вычисления количества автомобилей заданной марки.

2. Триггеры

В рамках лабораторной работы были созданы триггерные функции и триггеры для контроля операций с договорами проката, продлениями договоров, страховками и штрафами.

2.1. Запрет выдачи списанного автомобиля

SQL-команда создания триггерной функции.

```
CREATE OR REPLACE FUNCTION rental.check_car_not_written_off()  
RETURNS trigger  
LANGUAGE plpgsql  
AS $$  
DECLARE  
v_written_off CHAR(1);  
BEGIN  
SELECT c.written_off  
INTO v_written_off  
FROM rental.car c  
WHERE c.id = NEW.car_id;  
  
IF v_written_off IS NULL THEN  
RAISE EXCEPTION 'Автомобиль с id % не найден', NEW.car_id;  
END IF;  
  
IF v_written_off = 'Y' THEN  
RAISE EXCEPTION 'Списанный автомобиль нельзя выдать в прокат';  
END IF;  
  
RETURN NEW;  
END;  
$$;
```

SQL-команда создания триггера.

```
DROP TRIGGER IF EXISTS trg_check_car_not_written_off  
ON rental.rental_contract;  
  
CREATE TRIGGER trg_check_car_not_written_off  
BEFORE INSERT OR UPDATE OF car_id  
ON rental.rental_contract  
FOR EACH ROW  
EXECUTE FUNCTION rental.check_car_not_written_off();
```

Триггер `trg_check_car_not_written_off` запрещает выдачу автомобиля, если в таблице `rental.car` у него установлен признак списания `written_off = 'Y'`.

SQL-команда проверки триггера.

```
BEGIN;  
  
UPDATE rental.car  
SET
```

```
written_off = 'Y',
written_off_date = CURRENT_DATE
WHERE id = 4;

UPDATE rental.rental_contract
SET car_id = 4
WHERE id = 1;

ROLLBACK;
```

В результате проверки операция изменения договора была прервана ошибкой Списанный автомобиль нельзя выдать в прокат.

```
carsharing=# BEGIN;
BEGIN
carsharing=#
carsharing=# UPDATE rental.car
carsharing-*= SET
carsharing-*=      written_off = 'Y',
carsharing-*=      written_off_date = CURRENT_DATE
carsharing-*= WHERE id = 4;
UPDATE 1
carsharing=#
carsharing=# UPDATE rental.rental_contract
carsharing-*= SET car_id = 4
carsharing-*= WHERE id = 1;
ERROR:  Списанный автомобиль нельзя выдать в прокат
КОНТЕКСТ:  PL/pgSQL function check_car_not_written_off() line 15 at RAISE
carsharing=#
carsharing=# ROLLBACK;
ROLLBACK
carsharing=#
```

Рисунок 5 — Проверка работы триггера запрета выдачи списанного автомобиля.

2.2. Проверка занятости автомобиля

SQL-команда создания триггерной функции.

```
CREATE OR REPLACE FUNCTION rental.check_car_available()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
IF NEW.return_datetime IS NULL
AND NEW.contract_status IN ('открыт', 'просрочен') THEN

IF EXISTS (
SELECT 1
FROM rental.rental_contract rc
WHERE rc.car_id = NEW.car_id
AND rc.return_datetime IS NULL
AND rc.contract_status IN ('открыт', 'просрочен')
AND rc.id <> COALESCE(OLD.id, -1)
) THEN
RAISE EXCEPTION 'Автомобиль уже находится в незакрытом договоре';
END IF;

END IF;
```

```
RETURN NEW;  
END;  
$$;
```

SQL-команда создания триггера.

```
DROP TRIGGER IF EXISTS trg_check_car_available  
ON rental.rental_contract;  
  
CREATE TRIGGER trg_check_car_available  
BEFORE INSERT OR UPDATE OF car_id, return_datetime, contract_status  
ON rental.rental_contract  
FOR EACH ROW  
EXECUTE FUNCTION rental.check_car_available();
```

Триггер `trg_check_car_available` запрещает одновременную выдачу одного автомобиля по нескольким незакрытым договорам.

SQL-команда проверки триггера.

```
SELECT  
id,  
contract_number,  
car_id,  
return_datetime,  
contract_status  
FROM rental.rental_contract  
WHERE return_datetime IS NULL  
ORDER BY id;  
  
BEGIN;  
  
UPDATE rental.rental_contract  
SET car_id = 3  
WHERE id = 26;  
  
ROLLBACK;
```

В результате проверки операция изменения договора была прервана ошибкой Автомобиль уже находится в незакрытом договоре.

```

carsharing=# SELECT
carsharing=#     id,
carsharing=#     contract_number,
carsharing=#     car_id,
carsharing=#     return_datetime,
carsharing=#     contract_status
carsharing=# FROM rental.rental_contract
carsharing=# WHERE return_datetime IS NULL
carsharing=# ORDER BY id;
 id | contract_number | car_id | return_datetime | contract_status
-----+-----+-----+-----+-----
  3 | RC-2025-003    |    3 |                | открыт
 24 | RC-2026-024    |    3 |                | открыт
 25 | RC-2026-025    |    2 |                | просрочен
 26 | RC-2026-026    |    1 |                | открыт
 27 | RC-2026-027    |    8 |                | аннулирован
 28 | RC-2026-028    |    6 |                | просрочен
 29 | RC-2026-029    |    5 |                | открыт
 30 | RC-2026-030    |    7 |                | аннулирован
(8 строк)

carsharing=#
carsharing=# BEGIN;
BEGIN
carsharing=*#
carsharing=*# UPDATE rental.rental_contract
carsharing=*# SET car_id = 3
carsharing=*# WHERE id = 26;
ERROR: Автомобиль уже находится в незакрытом договоре
КОНТЕКСТ: PL/pgSQL function check_car_available() line 14 at RAISE
carsharing=!#
carsharing=!# ROLLBACK;
ROLLBACK
carsharing=#

```

Рисунок 6 — Проверка работы триггера контроля занятости автомобиля.

2.3. Проверка страховки при выдаче автомобиля

SQL-команда создания триггерной функции.

```

CREATE OR REPLACE FUNCTION rental.check_contract_insurance()
RETURNS trigger
LANGUAGE plpgsql
AS $$
DECLARE
v_car_model_id BIGINT;
v_insurance_model_id BIGINT;
v_start_date DATE;
v_end_date DATE;
BEGIN
IF NEW.insurance_id IS NULL THEN
RETURN NEW;
END IF;

SELECT c.model_id
INTO v_car_model_id
FROM rental.car c
WHERE c.id = NEW.car_id;

IF v_car_model_id IS NULL THEN
RAISE EXCEPTION 'Автомобиль с id % не найден', NEW.car_id;
END IF;

```

```

SELECT
i.model_id,
i.start_date,
i.end_date
INTO
v_insurance_model_id,
v_start_date,
v_end_date
FROM rental.insurance i
WHERE i.id = NEW.insurance_id;

IF v_insurance_model_id IS NULL THEN
RAISE EXCEPTION 'Страховка с id % не найдена', NEW.insurance_id;
END IF;

IF v_insurance_model_id <> v_car_model_id THEN
RAISE EXCEPTION 'Страховка не соответствует модели автомобиля';
END IF;

IF NEW.issue_datetime::date < v_start_date
OR (v_end_date IS NOT NULL AND NEW.issue_datetime::date > v_end_date) THEN
RAISE EXCEPTION 'Страховка не действует на дату выдачи автомобиля';
END IF;

RETURN NEW;
END;
$$;

```

SQL-команда создания триггера.

```

DROP TRIGGER IF EXISTS trg_check_contract_insurance
ON rental.rental_contract;

CREATE TRIGGER trg_check_contract_insurance
BEFORE INSERT OR UPDATE OF car_id, insurance_id, issue_datetime
ON rental.rental_contract
FOR EACH ROW
EXECUTE FUNCTION rental.check_contract_insurance();

```

Триггер `trg_check_contract_insurance` проверяет, что выбранная страховка соответствует модели автомобиля и действует на дату выдачи.

SQL-команда проверки триггера.

```

BEGIN;

UPDATE rental.rental_contract
SET
car_id = 4,
insurance_id = 3,
issue_datetime = '2025-06-01 10:00:00'
WHERE id = 1;

ROLLBACK;

```

В результате проверки операция изменения договора была прервана ошибкой Страховка не соответствует модели автомобиля.

```
carsharing=# BEGIN;
BEGIN
carsharing=#
carsharing=# UPDATE rental.rental_contract
carsharing-# SET
carsharing-#     car_id = 4,
carsharing-#     insurance_id = 3,
carsharing-#     issue_datetime = '2025-06-01 10:00:00'
carsharing-# WHERE id = 1;
ERROR: Страховка не соответствует модели автомобиля
КОНТЕКСТ: PL/pgSQL function check_contract_insurance() line 37 at RAISE
carsharing=!#
carsharing=!# ROLLBACK;
ROLLBACK
carsharing=#
```

Рисунок 7 — Проверка работы триггера контроля страховки при выдаче автомобиля.

2.4. Проверка корректности продления договора

SQL-команда создания триггерной функции.

```
CREATE OR REPLACE FUNCTION rental.check_contract_extension()
RETURNS trigger
LANGUAGE plpgsql
AS $$
DECLARE
v_contract_status VARCHAR(20);
BEGIN
IF NEW.extension_end <= NEW.extension_start THEN
RAISE EXCEPTION 'Дата окончания продления должна быть позже даты начала
продления';
END IF;

SELECT rc.contract_status
INTO v_contract_status
FROM rental.rental_contract rc
WHERE rc.id = NEW.contract_id;

IF v_contract_status IS NULL THEN
RAISE EXCEPTION 'Договор с id % не найден', NEW.contract_id;
END IF;

IF v_contract_status <> 'открыт' THEN
RAISE EXCEPTION 'Продление возможно только для открытого договора';
END IF;

RETURN NEW;
END;
$$;
```


SQL-команда создания триггера.

```
DROP TRIGGER IF EXISTS trg_check_contract_extension
ON rental.contract_extension;

CREATE TRIGGER trg_check_contract_extension
BEFORE INSERT OR UPDATE OF contract_id, extension_start, extension_end
ON rental.contract_extension
FOR EACH ROW
EXECUTE FUNCTION rental.check_contract_extension();
```

Триггер `trg_check_contract_extension` проверяет корректность продления договора проката: дата окончания продления должна быть позже даты начала, а сам договор должен иметь статус открыт.

SQL-команда проверки триггера.

```
BEGIN;

INSERT INTO rental.contract_extension (
contract_id,
extension_start,
extension_end,
reason
)
VALUES (
3,
'2025-06-10 12:00:00',
'2025-06-09 12:00:00',
'Некорректное тестовое продление'
);

ROLLBACK;

BEGIN;

INSERT INTO rental.contract_extension (
contract_id,
extension_start,
extension_end,
reason
)
VALUES (
25,
'2026-04-03 12:00:00',
'2026-04-05 12:00:00',
'Попытка продления просроченного договора'
);

ROLLBACK;
```

В результате проверки первая операция была прервана ошибкой Дата окончания продления должна быть позже даты начала продления, а вторая — ошибкой Продление возможно только для открытого договора.

```

carsharing=# BEGIN;
BEGIN
carsharing=#
carsharing=# INSERT INTO rental.contract_extension (
carsharing(*#      contract_id,
carsharing(*#      extension_start,
carsharing(*#      extension_end,
carsharing(*#      reason
carsharing(*# )
carsharing-# VALUES (
carsharing(*#      3,
carsharing(*#      '2025-06-10 12:00:00',
carsharing(*#      '2025-06-09 12:00:00',
carsharing(*#      'Некорректное тестовое продление'
carsharing(*# );
ERROR:  Дата окончания продления должна быть позже даты начала продления
КОНТЕКСТ:  PL/pgSQL function check_contract_extension() line 6 at RAISE
carsharing=!#
carsharing=!# ROLLBACK;
ROLLBACK
carsharing=#
carsharing=# BEGIN;
BEGIN
carsharing=#
carsharing=# INSERT INTO rental.contract_extension (
carsharing(*#      contract_id,
carsharing(*#      extension_start,
carsharing(*#      extension_end,
carsharing(*#      reason
carsharing(*# )
carsharing-# VALUES (
carsharing(*#      25,
carsharing(*#      '2026-04-03 12:00:00',
carsharing(*#      '2026-04-05 12:00:00',
carsharing(*#      'Попытка продления просроченного договора'
carsharing(*# );
ERROR:  Продление возможно только для открытого договора
КОНТЕКСТ:  PL/pgSQL function check_contract_extension() line 19 at RAISE
carsharing=!#
carsharing=!# ROLLBACK;
ROLLBACK
carsharing=#

```

Рисунок 8 — Проверка работы триггера контроля корректности продления договора.

2.5. Автоматическое закрытие договора при возврате автомобиля

SQL-команда создания триггерной функции.

```

CREATE OR REPLACE FUNCTION rental.set_contract_closed_on_return()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
IF NEW.return_datetime IS NOT NULL
AND OLD.return_datetime IS DISTINCT FROM NEW.return_datetime THEN

NEW.return_mark = 'Y';
NEW.contract_status = 'закрыт';

END IF;

RETURN NEW;

```

```
END;  
$$;
```

SQL-команда создания триггера.

```
DROP TRIGGER IF EXISTS trg_set_contract_closed_on_return  
ON rental.rental_contract;  
  
CREATE TRIGGER trg_set_contract_closed_on_return  
BEFORE UPDATE OF return_datetime  
ON rental.rental_contract  
FOR EACH ROW  
EXECUTE FUNCTION rental.set_contract_closed_on_return();
```

Триггер `trg_set_contract_closed_on_return` автоматически устанавливает отметку о возврате и закрывает договор при заполнении поля `return_datetime`.

SQL-команда проверки триггера.

```
BEGIN;  
  
SELECT  
id,  
contract_number,  
return_datetime,  
return_mark,  
contract_status  
FROM rental.rental_contract  
WHERE id = 3;  
  
UPDATE rental.rental_contract  
SET return_datetime = '2025-05-25 18:00:00'  
WHERE id = 3;  
  
SELECT  
id,  
contract_number,  
return_datetime,  
return_mark,  
contract_status  
FROM rental.rental_contract  
WHERE id = 3;  
  
ROLLBACK;
```

В результате проверки после изменения поля `return_datetime` у договора автоматически изменились поля `return_mark` и `contract_status`.

```

carsharing=# BEGIN;
BEGIN
carsharing=#
carsharing=# SELECT
carsharing=#     id,
carsharing=#     contract_number,
carsharing=#     return_datetime,
carsharing=#     return_mark,
carsharing=#     contract_status
carsharing=# FROM rental.rental_contract
carsharing=# WHERE id = 3;
 id | contract_number | return_datetime | return_mark | contract_status
-----+-----+-----+-----+-----
  3 | RC-2025-003    |                | N           | открыт
(1 строка)

carsharing=#
carsharing=# UPDATE rental.rental_contract
carsharing=# SET return_datetime = '2025-05-25 18:00:00'
carsharing=# WHERE id = 3;
UPDATE 1
carsharing=#
carsharing=# SELECT
carsharing=#     id,
carsharing=#     contract_number,
carsharing=#     return_datetime,
carsharing=#     return_mark,
carsharing=#     contract_status
carsharing=# FROM rental.rental_contract
carsharing=# WHERE id = 3;
 id | contract_number | return_datetime | return_mark | contract_status
-----+-----+-----+-----+-----
  3 | RC-2025-003    | 2025-05-25 18:00:00 | Y           | закрыт
(1 строка)

carsharing=#
carsharing=# ROLLBACK;
ROLLBACK
carsharing=#

```

Рисунок 9 — Проверка работы триггера автоматического закрытия договора при возврате автомобиля.

2.6. Автоматическое определение плательщика штрафа

SQL-команда создания триггерной функции.

```

CREATE OR REPLACE FUNCTION rental.set_fine_payer()
RETURNS trigger
LANGUAGE plpgsql
AS $$
DECLARE
v_deposit_returned_full CHAR(1);
v_deposit_returned_partial CHAR(1);
BEGIN
SELECT
rc.deposit_returned_full,
rc.deposit_returned_partial
INTO
v_deposit_returned_full,
v_deposit_returned_partial
FROM rental.rental_contract rc
WHERE rc.id = NEW.contract_id;

```

```

IF v_deposit_returned_full IS NULL THEN
RAISE EXCEPTION 'Договор с id % не найден', NEW.contract_id;
END IF;

IF v_deposit_returned_full = 'Y'
OR v_deposit_returned_partial = 'Y' THEN
NEW.payer = 'арендодатель';
ELSE
NEW.payer = 'клиент';
END IF;

RETURN NEW;
END;
$$;

```

SQL-команда создания триггера.

```

DROP TRIGGER IF EXISTS trg_set_fine_payer
ON rental.contract_fine;

CREATE TRIGGER trg_set_fine_payer
BEFORE INSERT OR UPDATE OF contract_id
ON rental.contract_fine
FOR EACH ROW
EXECUTE FUNCTION rental.set_fine_payer();

```

Триггер `trg_set_fine_payer` автоматически определяет плательщика штрафа. Если по договору залог уже возвращён полностью или частично, плательщиком становится арендодатель, иначе — клиент.

SQL-команда проверки триггера.

```

BEGIN;

INSERT INTO rental.contract_fine (
contract_id,
accident_id,
violation_datetime,
violation_type_id,
fine_amount,
payment_amount,
payment_status,
payment_date,
payer
)
VALUES (
1,
NULL,
'2025-04-10 12:00:00',
3,
500.00,
0.00,
'начислен',
NULL,
NULL
)

```

```
);

SELECT
id,
contract_id,
violation_datetime,
violation_type_id,
fine_amount,
payment_status,
payer
FROM rental.contract_fine
WHERE contract_id = 1
ORDER BY id DESC
LIMIT 1;

ROLLBACK;
```

В результате проверки при добавлении штрафа по договору с возвращённым залогом поле payer было автоматически заполнено значением арендодатель.

```
carsharing=# BEGIN;
BEGIN
carsharing=#
carsharing=# INSERT INTO rental.contract_fine (
carsharing(*#      contract_id,
carsharing(*#      accident_id,
carsharing(*#      violation_datetime,
carsharing(*#      violation_type_id,
carsharing(*#      fine_amount,
carsharing(*#      payment_amount,
carsharing(*#      payment_status,
carsharing(*#      payment_date,
carsharing(*#      payer
carsharing(*# )
carsharing-!# VALUES (
carsharing(*#      1,
carsharing(*#      NULL,
carsharing(*#      '2025-04-10 12:00:00',
carsharing(*#      3,
carsharing(*#      500.00,
carsharing(*#      0.00,
carsharing(*#      'начислен',
carsharing(*#      NULL,
carsharing(*#      NULL
carsharing(*# );
ERROR:  Дата нарушения не может быть позже даты возврата автомобиля
КОНТЕКСТ:  PL/pgSQL function check_fine_period() line 25 at RAISE
carsharing=!#
carsharing=!# SELECT
carsharing-!#      id,
carsharing-!#      contract_id,
carsharing-!#      violation_datetime,
carsharing-!#      violation_type_id,
carsharing-!#      fine_amount,
carsharing-!#      payment_status,
carsharing-!#      payer
carsharing-!# FROM rental.contract_fine
carsharing-!# WHERE contract_id = 1
carsharing-!# ORDER BY id DESC
carsharing-!# LIMIT 1;
ERROR:  current transaction is aborted, commands ignored until end of transaction block
carsharing=!#
carsharing=!# ROLLBACK;
ROLLBACK
carsharing=#
```

Рисунок 10 — Проверка работы триггера автоматического определения плательщика штрафа.

2.7. Проверка даты нарушения по договору

SQL-команда создания триггерной функции.

```
CREATE OR REPLACE FUNCTION rental.check_fine_period()
RETURNS trigger
LANGUAGE plpgsql
AS $$
DECLARE
v_issue_datetime TIMESTAMP;
v_return_datetime TIMESTAMP;
BEGIN
SELECT
rc.issue_datetime,
rc.return_datetime
INTO
v_issue_datetime,
v_return_datetime
FROM rental.rental_contract rc
WHERE rc.id = NEW.contract_id;

IF v_issue_datetime IS NULL THEN
RAISE EXCEPTION 'Договор с id % не найден', NEW.contract_id;
END IF;

IF NEW.violation_datetime < v_issue_datetime THEN
RAISE EXCEPTION 'Дата нарушения не может быть раньше даты выдачи
автомобиля';
END IF;

IF v_return_datetime IS NOT NULL
AND NEW.violation_datetime > v_return_datetime THEN
RAISE EXCEPTION 'Дата нарушения не может быть позже даты возврата
автомобиля';
END IF;

RETURN NEW;
END;
$$;
```

SQL-команда создания триггера.

```
DROP TRIGGER IF EXISTS trg_check_fine_period
ON rental.contract_fine;

CREATE TRIGGER trg_check_fine_period
BEFORE INSERT OR UPDATE OF contract_id, violation_datetime
ON rental.contract_fine
FOR EACH ROW
EXECUTE FUNCTION rental.check_fine_period();
```

Триггер `trg_check_fine_period` проверяет, что дата нарушения находится в пределах периода действия договора проката.

SQL-команда проверки триггера.

```
SELECT
id,
contract_number,
issue_datetime,
return_datetime,
contract_status
FROM rental.rental_contract
WHERE id = 1;

BEGIN;

INSERT INTO rental.contract_fine (
contract_id,
accident_id,
violation_datetime,
violation_type_id,
fine_amount,
payment_amount,
payment_status,
payment_date,
payer
)
VALUES (
1,
NULL,
'2000-01-01 12:00:00',
3,
500.00,
0.00,
'начислен',
NULL,
NULL
);

ROLLBACK;
```

В результате проверки операция добавления штрафа была прервана ошибкой Дата нарушения не может быть раньше даты выдачи автомобиля.


```

carsharing=# SELECT
carsharing=#     id,
carsharing=#     contract_number,
carsharing=#     issue_datetime,
carsharing=#     return_datetime,
carsharing=#     contract_status
carsharing=# FROM rental.rental_contract
carsharing=# WHERE id = 1;
 id | contract_number | issue_datetime   | return_datetime | contract_status
-----+-----+-----+-----+-----
  1 | RC-2025-001    | 2025-03-10 10:00:00 | 2025-03-15 18:00:00 | закрыт
(1 строка)

carsharing=#
carsharing=# BEGIN;
BEGIN
carsharing=#
carsharing=# INSERT INTO rental.contract_fine (
carsharing(*#      contract_id,
carsharing(*#      accident_id,
carsharing(*#      violation_datetime,
carsharing(*#      violation_type_id,
carsharing(*#      fine_amount,
carsharing(*#      payment_amount,
carsharing(*#      payment_status,
carsharing(*#      payment_date,
carsharing(*#      payer
carsharing(*# )
carsharing-# VALUES (
carsharing(*#      1,
carsharing(*#      NULL,
carsharing(*#      '2000-01-01 12:00:00',
carsharing(*#      3,
carsharing(*#      500.00,
carsharing(*#      0.00,
carsharing(*#      'начислен',
carsharing(*#      NULL,
carsharing(*#      NULL
carsharing(*# );
ERROR:  Дата нарушения не может быть раньше даты выдачи автомобиля
КОНТЕКСТ:  PL/pgSQL function check_fine_period() line 20 at RAISE
carsharing=#
carsharing=# ROLLBACK;
ROLLBACK
carsharing=#

```

Рисунок 11 — Проверка работы триггера контроля даты нарушения по договору.

3. Итоговая проверка созданных объектов

После создания процедур, триггерных функций и триггеров была выполнена итоговая проверка объектов в схеме rental.

SQL-команда проверки процедур.

```

SELECT
n.nspname AS schema_name,
p.proname AS procedure_name,
p.prokind AS object_type
FROM pg_proc p
JOIN pg_namespace n
ON n.oid = p.pronamespace
WHERE n.nspname = 'rental'
AND p.proname IN (
'write_off_cars_before_year',
'issue_car_with_discount',
'count_cars_by_brand'
)
ORDER BY p.proname;

```

SQL-команда проверки триггерных функций.

```
SELECT
n.nspname AS schema_name,
p.proname AS function_name,
p.prokind AS object_type
FROM pg_proc p
JOIN pg_namespace n
ON n.oid = p.pronamespace
WHERE n.nspname = 'rental'
AND p.proname IN (
'check_car_not_written_off',
'check_car_available',
'check_contract_insurance',
'check_contract_extension',
'set_contract_closed_on_return',
'set_fine_payer',
'check_fine_period'
)
ORDER BY p.proname;
```

SQL-команда проверки триггеров.

```
SELECT
trigger_schema,
trigger_name,
event_object_table,
action_timing,
event_manipulation
FROM information_schema.triggers
WHERE trigger_schema = 'rental'
AND trigger_name IN (
'trg_check_car_not_written_off',
'trg_check_car_available',
'trg_check_contract_insurance',
'trg_check_contract_extension',
'trg_set_contract_closed_on_return',
'trg_set_fine_payer',
'trg_check_fine_period'
)
ORDER BY
event_object_table,
trigger_name,
event_manipulation;
```

В результате проверки были выведены созданные процедуры, триггерные функции и триггеры. В таблице процедур значение object_type = p соответствует процедурам, а в таблице функций значение object_type = f соответствует функциям.

```

carsharing=#
carsharing=# SELECT
carsharing=#     n.nspname AS schema_name,
carsharing=#     p.proname AS procedure_name,
carsharing=#     p.prokind AS object_type
carsharing=# FROM pg_proc p
carsharing=# JOIN pg_namespace n
carsharing=#     ON n.oid = p.pronamespace
carsharing=# WHERE n.nspname = 'rental'
carsharing=#     AND p.proname IN (
carsharing=#         'write_off_cars_before_year',
carsharing=#         'issue_car_with_discount',
carsharing=#         'count_cars_by_brand'
carsharing=#     )
carsharing=# ORDER BY p.proname;
 schema_name |      procedure_name      | object_type
-----
rental       | count_cars_by_brand      | p
rental       | issue_car_with_discount  | p
rental       | write_off_cars_before_year | p
(3 строки)

carsharing=#
carsharing=# SELECT
carsharing=#     n.nspname AS schema_name,
carsharing=#     p.proname AS function_name,
carsharing=#     p.prokind AS object_type
carsharing=# FROM pg_proc p
carsharing=# JOIN pg_namespace n
carsharing=#     ON n.oid = p.pronamespace
carsharing=# WHERE n.nspname = 'rental'
carsharing=#     AND p.proname IN (
carsharing=#         'check_car_not_written_off',
carsharing=#         'check_car_available',
carsharing=#         'check_contract_insurance',
carsharing=#         'check_contract_extension',
carsharing=#         'set_contract_closed_on_return',
carsharing=#         'set_fine_payer',
carsharing=#         'check_fine_period'
carsharing=#     )
carsharing=# ORDER BY p.proname;
 schema_name |      function_name      | object_type
-----
rental       | check_car_available      | f
rental       | check_car_not_written_off | f
rental       | check_contract_extension | f
rental       | check_contract_insurance | f
rental       | check_fine_period        | f
rental       | set_contract_closed_on_return | f
rental       | set_fine_payer           | f
(7 строк)

carsharing=#
carsharing=# SELECT
carsharing=#     trigger_schema,
carsharing=#     trigger_name,
carsharing=#     event_object_table,
carsharing=#     action_timing,
carsharing=#     event_manipulation
carsharing=# FROM information_schema.triggers
carsharing=# WHERE trigger_schema = 'rental'
carsharing=#     AND trigger_name IN (
carsharing=#         'trg_check_car_not_written_off',
carsharing=#         'trg_check_car_available',
carsharing=#         'trg_check_contract_insurance',
carsharing=#         'trg_check_contract_extension',
carsharing=#         'trg_set_contract_closed_on_return',
carsharing=#         'trg_set_fine_payer',
carsharing=#         'trg_check_fine_period'
carsharing=#     )
carsharing=# ORDER BY
carsharing=#     event_object_table,
carsharing=#     trigger_name,
carsharing=#     event_manipulation;
 trigger_schema |      trigger_name      | event_object_table | action_timing | event_manipulation
-----
rental          | trg_check_contract_extension | contract_extension | BEFORE        | INSERT
rental          | trg_check_contract_extension | contract_extension | BEFORE        | UPDATE
rental          | trg_check_fine_period        | contract_fine      | BEFORE        | INSERT
rental          | trg_check_fine_period        | contract_fine      | BEFORE        | UPDATE
rental          | trg_set_fine_payer           | contract_fine      | BEFORE        | INSERT
rental          | trg_set_fine_payer           | contract_fine      | BEFORE        | UPDATE
rental          | trg_check_car_available      | rental_contract    | BEFORE        | INSERT
rental          | trg_check_car_available      | rental_contract    | BEFORE        | UPDATE
rental          | trg_check_car_not_written_off | rental_contract    | BEFORE        | INSERT
rental          | trg_check_car_not_written_off | rental_contract    | BEFORE        | UPDATE
rental          | trg_check_contract_insurance | rental_contract    | BEFORE        | INSERT
rental          | trg_check_contract_insurance | rental_contract    | BEFORE        | UPDATE
rental          | trg_set_contract_closed_on_return | rental_contract    | BEFORE        | UPDATE
(13 строк)

```

Рисунок 12 — Итоговая проверка созданных процедур, триггерных функций и триггеров.

Выводы

В ходе выполнения лабораторной работы №5 были созданы хранимые процедуры, триггерные функции и триггеры в базе данных PostgreSQL.

Были реализованы 3 процедуры для индивидуальной базы данных варианта 12 «Прокат автомобилей»: процедура списания автомобилей, выпущенных ранее заданного года, процедура выдачи автомобиля с расчётом стоимости проката с учётом скидки постоянного клиента, а также процедура вычисления количества автомобилей заданной марки.

Также были созданы 7 триггеров для контроля бизнес-правил предметной области. Реализованные триггеры запрещают выдачу списанного автомобиля, проверяют занятость автомобиля, контролируют соответствие страховки выбранному автомобилю, проверяют корректность продления договора, автоматически закрывают договор при возврате автомобиля, определяют плательщика штрафа и проверяют дату нарушения по договору.

Работа созданных процедур и триггеров была проверена в консоли SQL Shell psql с использованием тестовых операций и транзакций BEGIN / ROLLBACK. Результаты проверки подтвердили корректность работы созданных объектов базы данных.